

✓ Part 2: Think Like a Developer

Part 1 was more focused on becoming someone who understands code and writing pieces of code. These pieces of code however do not work in the real world. Being just a programmer is ideal for cases where just snippets of code are needed and not full fledged applications. In the real world of software engineering there is need of at least a developer. A developer builds applications and software products. This is broader than programming.

Focus is on:

- building software solutions,
- user requirements,
- application functionality.

A developer asks:

"How do I build this application?"

Examples of developers include:

- Web Developer
- Backend Developer
- Mobile App Developer
- Game Developer

So in this section we improve the skills you have acquired as a programmer and add more. We start with Python itself. Moving from beginner to intermediate stage. We also focus on some important engineering essentials needed by every developer.

✓ Functional Techniques in Python



List comprehension

concise way to create new lists by applying an expression to each item in an existing iterable like a list, tuple or range.

It helps to write clean, readable and efficient code compared to traditional loops.

Syntax [expression for item in iterable if condition]

Parameters:

- expression: operation or value to include in the new list.
- item: current element from the iterable.
- iterable: sequence like a list, tuple or range.
- if condition (optional): filter to include only items that satisfy the condition.

```
a = [2, 3, 4, 5]
res = [val*2 for val in a]
print(res)
# Explanation: res = [val*2 for val in a] use list comprehension to create a new list by doubling each number
```

```
[4, 6, 8, 10]
```

```
# One where there is a conditional present.
```

```
a = [1, 2, 3, 4, 5]
res = [val for val in a if val % 2 == 0]
print(res)
```

```
[4, 8]
```

```
# Example 2: Here, list comprehension is used with a condition to create a new list with numbers greater than 10

b = [5, 12, 7, 18, 3, 20]
res = [val for val in b if val > 10]
print(res)
```

```
[12, 18, 20]
```

```
# 1. Creating a list from a range: One can quickly create a list of numbers within a specific range using list comprehension.
```

```
a = [i for i in range(10)]
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

✓ Lambda functions

small anonymous functions Do not have a defined name. These are small, short-lived functions used to pass simple logic to another function.

- Contain only one expression.
- Result of that expression is returned automatically (no return keyword needed). In this example, a lambda function is defined to convert a string to its upper case using upper().

Syntax function_name = lambda argument: expression

```
a = 'Convert this string to upper characters only'
upper = lambda x: x.upper()
print(upper(a))
```

```
CONVERT THIS STRING TO UPPER CHARACTERS ONLY
```

Use Cases

1. Condition Checking: lambda function can use conditional expressions (if-else) to return different results based on a condition.
2. List Comprehension: Lambda can be combined with list comprehensions to apply the same operation to multiple values in a compact way.

✓ Map

map() function in Python applies a given function to each element of an iterable (list, tuple, set, etc.) and returns a map object (iterator). It is a higher-order function used for uniform element-wise transformations, enabling concise and efficient code.

Syntax map(function, iterable,..) Parameters:

- function: The function to apply to every element of the iterable.
- iterable: One or more iterable objects (list, tuple, etc.) whose elements will be processed. Note: You can pass multiple iterables if the function accepts multiple arguments.

```
# Let's start with a simple example of using map() to convert a list of strings into a list of integers.
```

```
s = ['1', '2', '3', '4']  
res = map(int, s)  
print(list(res))
```

```
[1, 2, 3, 4]
```

```
# Double items example
```

```
def double(val):  
    return val * 2  
  
a = [1, 2, 3, 4]  
res = list(map(double, a))  
print(res)
```

```
[2, 4, 6, 8]
```

✓ Filter

`filter()` function is used to extract elements from an iterable (like a list, tuple or set) that satisfy a given condition. It works by applying a function to each element and keeping only those for which function returns `True`.

```
# Example: This example shows how to keep only the words starting with the letter 'a' from a list of fruits
def starts_a(w):
    return w.startswith("a")

li = ["apple", "banana", "avocado", "cherry", "apricot"]
res = filter(starts_a, li)
print(list(res))

['apple', 'avocado', 'apricot']
```

✓ Reduce

`reduce()` function (from the `functools` module) applies a function cumulatively to the elements of an iterable and returns a single final value. It processes elements step-by-step, combining two elements at a time until only one result remains. Example: In this example, `reduce()` combines all strings in a list into one sentence.

Syntax `reduce(function, iterable[, initializer])` Parameters:

- `function`: A function that takes two arguments and returns a single value.
- `iterable`: The sequence to be reduced (list, tuple, etc.).
- `initializer` (optional): A starting value that is placed before first element. Returns: Returns a single final value after processing all elements.

```
from functools import reduce
a = ["Become", "one", "statement"]
r = reduce(lambda x, y: x + " " + y, a)
print(r)
```

Become one statement

```
import functools
import operator
a = [1, 3, 5, 6, 2]
print(functools.reduce(operator.add, a))
print(functools.reduce(operator.mul, a))
print(functools.reduce(operator.add, ["Become", "one", "statement"]))
```

```
17
180
Becomeonestatement
```

✓ Accumulate

`accumulate()` function (from `itertools`) and `reduce()` both apply a function cumulatively to items in a sequence. However, `accumulate()` returns an iterator of intermediate results, while `reduce()` returns only final value.

Example: This code demonstrates how `accumulate()` from `itertools` module works it performs cumulative operations and returns all intermediate results instead of just a single final value.

```
from itertools import accumulate
from operator import add

a = [1, 2, 3, 4, 5]
res = accumulate(a, add)
print(list(res))
```

```
[1, 3, 6, 10, 15]
```

✓ Regular Expressions

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

RegEx can be used to check if a string contains the specified search pattern.

RegEx Module Python has a built-in package called re, which can be used to work with Regular Expressions.

```
import re
```

```
import re

txt = "The rain in Gokwe"
x = re.search("^The.*Gokwe$", txt)
print(x)

<re.Match object; span=(0, 17), match='The rain in Gokwe'>
```

Common functions

Category	Pattern	Meaning	Example
Import	<code>import re</code>	Import the regular expressions (regex) module	<code>import re</code>
Search	<code>re.search()</code>	Find the first match in a string	<code>re.search(r"\d+", text)</code>
Find All	<code>re.findall()</code>	Find all matches in a string	<code>re.findall(r"\w+", text)</code>
Replace	<code>re.sub()</code>	Replace matching text with new text	<code>re.sub(r"\s+", " ", text)</code>
Split	<code>re.split()</code>	Split a string using a regex pattern	<code>re.split(r"[,;]", text)</code>
Compile	<code>re.compile()</code>	Compile a regex pattern for repeated use	<code>pattern = re.compile(r"\d+")</code>

Character classes

Pattern	Meaning
<code>.</code>	Matches any character except a newline (by default)
<code>\d</code>	Matches any digit (<code>0-9</code>)
<code>\D</code>	Matches any non-digit character
<code>\w</code>	Matches any word character (<code>a-z</code> , <code>A-Z</code> , <code>0-9</code> , <code>_</code>)

Pattern	Meaning
<code>\W</code>	Matches any non-word character
<code>\s</code>	Matches any whitespace character (space, tab, newline, etc.)
<code>\S</code>	Matches any non-whitespace character

Character Sets

Pattern	Meaning
<code>[abc]</code>	Matches any one character: <code>a</code> , <code>b</code> , or <code>c</code>
<code>[^abc]</code>	Matches any character except <code>a</code> , <code>b</code> , or <code>c</code>
<code>[a-z]</code>	Matches any lowercase letter (<code>a</code> to <code>z</code>)
<code>[A-Z]</code>	Matches any uppercase letter (<code>A</code> to <code>Z</code>)
<code>[0-9]</code>	Matches any digit (<code>0</code> to <code>9</code>)

Character groups

Pattern	Meaning
<code>()</code>	Capturing group (captures matched text for later use or backreferences)
<code>(?:)</code>	Non-capturing group (groups expressions without capturing the match)
<code> </code>	Alternation (logical OR ; matches either the expression on the left or the right, e.g., <code>cat dog</code>)

Quantifiers

Pattern	Meaning
<code>*</code>	Matches 0 or more occurrences of the preceding character or group
<code>+</code>	Matches 1 or more occurrences of the preceding character or group
<code>?</code>	Matches 0 or 1 occurrence (makes the preceding character or group optional)
<code>{n}</code>	Matches exactly <i>n</i> occurrences of the preceding character or group
<code>{n,m}</code>	Matches between <i>n</i> and <i>m</i> occurrences of the preceding character or group

Anchors

Pattern	Meaning
<code>^</code>	Matches the start of a string (or the start of a line in multiline mode)
<code>\$</code>	Matches the end of a string (or the end of a line in multiline mode)
<code>\b</code>	Matches a word boundary (the position between a word character and a non-word character)

Flags

Flag	Meaning
<code>re.I</code> (<code>re.IGNORECASE</code>)	Performs case-insensitive matching (ignores letter case)
<code>re.M</code> (<code>re.MULTILINE</code>)	Makes <code>^</code> and <code>\$</code> match the start and end of each line, not just the entire string
<code>re.S</code> (<code>re.DOTALL</code>)	Makes the dot (<code>.</code>) match newline characters as well

Common Examples

Purpose	Regex Pattern
Email Address	<code>^[\\w\\.\\-]+@[\\w\\.\\-]+\\.\\w+\$</code>
URL	<code>https?://\\S+</code>
Number	<code>\\d+</code>
Word	<code>\\b\\w+\\b</code>
Zimbabwe Phone Number	<code>\\+263\\d{9}</code>
Remove Extra Spaces	<code>\\s+</code>
Password Validation	<code>^(?=.*[A-Z])(?=.*\\d){8,}\$</code> (At least 8 characters, one uppercase letter, and one digit)

✓ Object Oriented Programming

programming paradigm that organizes programs around objects rather than functions.

An object combines:

Data (Attributes) - information about the object. Behavior (Methods) - actions the object can perform.

Class

A class is a blueprint or template used to create objects.

It defines: What data objects will store. What actions objects can perform.

Syntax `class ClassName: pass`

Object

An object is an instance of a class.

Attributes

Attributes are variables that belong to an object.

For example, a student may have a name, age, programme, and registration number.

Methods

Methods are functions defined inside a class.

```
class Student:
    def greet(self):
        print("Hello", self.name)
```

The Constructor (**init**)

The constructor is a special method that automatically runs whenever an object is created.

Syntax: `def init(self):`

E.g. `class Student:
 def init(self):
 self.name = "John"
 self.age = 20`

you can also initialise and object with values `def init(self, name, age):
 self.name = name
 self.age = age`

The self Keyword

`self` refers to the current object.

Whenever you access an object's variables or methods, you use `self`.

The str() Method

str method allows us to define a custom string representation of an object. By default, when we print an object or convert it to a string using str(). Take an example of using str() method to provide a readable string output for an object.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def __str__(self):
        return f"{self.owner} has a balance of ${self.balance}"

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

    def display(self):
        print(self.owner)
        print("Balance:", self.balance)
```

```
# Using the class

account = BankAccount("John", 500)

account.deposit(300)
account.withdraw(100)

account.display()

print(account)
```

```
John  
Balance: 700  
John has a balance of $700
```

Inheritance

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class).

Here, a parent class Animal is created that has a method info(). Then a child classes Dog is created that inherit from Animal and add their own behavior.

```
class Animal: def init(self, name): self.name = name
```

```
def info(self):  
    print("Animal name:", self.name)
```

```
class Dog(Animal): def sound(self): print(self.name, "barks")
```

```
d = Dog("Buddy")
```

Inherited method

```
d.info()
```

```
d.sound()
```

super() Used to call parent method/attribute when child method/attribute also exists In Python, super() function is used to call methods from a parent (superclass) inside a child (subclass). It allows to extend or override inherited methods while still reusing the parent's functionality.

```
class Parent: def show(self): print("This is Parent class method")
```

```
class Child(Parent): def show(self): super().show() # Calling parent method print("This is Child class method")
```

```
obj = Child() obj.show()
```

Encapsulation

Hiding implementation details

- The idea of encapsulation is to bind the data members and methods into a single unit.
- Helps in better maintainability, readability and usability as we do not need to explicitly pass data members to member methods
- Helps maintain data integrity by allowing validation and control over the values assigned to variables.
- It helps in achieving abstraction. A class can hide the implementation part and discloses only the functionalities required by other classes which allows later changes to representations or implementations without impacting the codes that uses this class.

This example shows encapsulation by keeping `__salary` variable private inside Employee class. It cannot be accessed directly from outside the class.

Some attributes are private and some public (access specifiers)

```
class Employee:
    def init(self, name, salary):
        self.name = name # public attribute
        self.__salary = salary # private attribute

emp = Employee("Fedrick", 50000)
print(emp.name)
print(emp.__salary) # will give an error because private attribute cannot be accessed directly
```

Access Specifiers Access specifiers define how class members (variables and methods) can be accessed from outside the class. They help in implementing encapsulation by controlling the visibility of data. There are three types of access specifiers:

1. Public Accessed from anywhere
- ```
class Employee:
 def init(self, name):
 self.name = name # public attribute
 def display_name(self):
 # public method
 print(self.name)
```

```
emp = Employee("John")
emp.display_name() # Accessible
print(emp.name) # Accessible
```

2. Protected Accessed by a class and its subclasses Add one underscore before method/attribute name

```
class Employee: def init(self, name, age): self.name = name # public self._age = age # protected
class SubEmployee(Employee): def show_age(self): print("Age:", self._age) # Accessible in subclass
emp = SubEmployee("Ross", 30) print(emp.name) # Public accessible emp.show_age() # Protected accessed through subclass
```

3. Private Accessed within class only Add double underscore

```
class Employee: def init(self, name, salary): self.name = name # public self.__salary = salary # private
```

```
def show_salary(self):
 print("Salary:", self.__salary)
```

```
emp = Employee("Robert", 60000) print(emp.name) # Public accessible emp.show_salary() # Accessing private correctly
```

## print(emp.\_\_salary) # Error: Not accessible directly

### Getter and Setter Methods

In Python, getter and setter methods are used to access and modify private attributes safely. Instead of accessing private data directly, these methods provide controlled access, allowing you to:

- Read data using a getter method.
- Update data using a setter method with optional validation or restrictions. Example: This example shows how to use a getter and a setter method to safely access and update a private attribute (\_\_salary).

```
class Employee: def init(self): self.__salary = 50000 # Private attribute
```

```
def get_salary(self): # Getter method
 return self.__salary

def set_salary(self, amount): # Setter method
 if amount > 0:
 self.__salary = amount
```

```
else:
 print("Invalid salary amount!")
```

```
emp = Employee() print(emp.get_salary()) # Access salary using getter
emp.set_salary(60000) # Update salary using setter print(emp.get_salary())
```

## Polymorphism

Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context. Technically, Polymorphism allows same method, function or operator to behave differently depending on object it is working with. This makes code more flexible and reusable. Real Life Example: In a backend payment system, multiple payment options are available such as Credit Card, UPI, NetBanking and Wallet. All payment types use a common method named `processPayment()` but different implementations:

2 types:

1. Compile time - method overloading Python does not support this
2. Runtime - method overriding class Animal: `def sound(self): return "Some generic sound"`

```
class Dog(Animal):
 def sound(self):
 return "Bark"
```

```
class Cat(Animal):
 def sound(self):
 return "Meow"
```

## Polymorphic behavior

```
animals = [Dog(), Cat(), Animal()]
for animal in animals:
 print(animal.sound())
```

Polymorphism in built in functions `print(len("Hello"))` # String length `print(len([1, 2, 3]))` # List length

`print(max(1, 3, 2))` # Maximum of integers `print(max("a", "z", "m"))` # Maximum in strings

Metaclass A metaclass is a class that defines how other classes are created and behave. Just like objects are created from classes, classes themselves are created from metaclasses. In short:

- Class: creates objects
- Metaclass: creates classes By default, Python uses the built-in metaclass "type" to create all classes.

```
class Meta(type):
 def new(cls, name, bases, dct):
 dct['greet'] = lambda self: f"Hello from {name}!"
 return super().new(cls, name, bases, dct)
```

```
class Person(metaclass=Meta):
 def init(self, name):
 self.name = name
```

```
p = Person("Olivia")
```

```
print(p.greet())
```

## ✓ Closures and Decorators

### ✓ Closures

Closures in Python are like “memory-equipped” functions. They allow a function to remember values from the environment in which it was created even if that environment no longer exists. Closures are used in functional programming, event handling and callback functions where you need to retain some state without using global variables.

A closure is formed when:

- A function is defined inside another function (nested function).
- The inner function references variables from the outer function.
- The outer function returns the inner function.

```
def outer_function(x):
 # Outer function: takes 'x' and defines inner_function
 def inner_function(y):
```



```
 return x + y # 'x' is remembered from outer_function
 return inner_function # Returns inner function (closure)

Create a closure with x = 10
closure = outer_function(10)

Call the closure with different values of 'y'
print(closure(5))
print(closure(20))
```

```
15
30
```

?? Whats the point?

- Function Factories: Functions that generate other functions with pre-set behavior.
- Data Hiding: Protecting variables from being accessed directly.
- Event Handling: Remembering state between user interactions.
- Callbacks: Functions passed around while keeping context.

## ✓ Decorators

Decorators are flexible way to modify or extend behavior of functions or methods, without changing their actual code.

- A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.
- They are often used in scenarios such as logging, authentication and memoization, allowing us to add additional functionality to existing functions or methods in a clean, reusable way.

```
def decorator(func):
 def wrapper():
 print("Before calling the function.")
 func()
```

```
 print("After calling the function.")
 return wrapper

@decorator
def greet():
 print("Hello, World!")
greet()
```

```
Before calling the function.
Hello, World!
After calling the function.
```

## Monkey Patching

In Python, Monkey Patching refers to dynamically modifying or extending behavior of a class or module at runtime. This allows developers to change how functions, methods or classes behave without altering original source code. Monkey patching is a powerful feature, but it should be used carefully because it can lead to unexpected behavior if not managed properly. For Example: Suppose we have a module monk.py with a simple class:

### monk.py

```
class A:
 def func(self):
 print("func() is being called")
```

## Then we can change behaviour at runtime

```
import monk
```

## New function to replace original func()

```
def monkey_f(self):
 print("monkey_f() is being called")
```

## Replacing the address of "func" with "monkey\_f"

```
monk.A.func = monkey_f obj = monk.A() obj.func()
```

### Use Cases

- Fixing Bugs in Third-Party Modules: Methods can be patched without modifying original library.
- Testing and Mocking: Methods can be replaced during unit testing to simulate specific behaviors.
- Dynamic Behavior in Frameworks: Some frameworks, such as Django, use monkey patching internally to extend classes dynamically.

## ✓ Memory Management

## ✓ Iterators

An iterator in Python is an object used to traverse through all the elements of a collection (like lists, tuples or dictionaries) one element at a time. It follows the iterator protocol, which involves two key methods:

- **iter()**: Returns the iterator object itself.
- **next()**: Returns the next value from the sequence. Raises StopIteration when the sequence ends.

```
s = "GSQ"
it = iter(s)

print(next(it))
print(next(it))
print(next(it))
```

```
G
S
Q
```

## Generators

A generator function is a special type of function that returns an iterator object. Instead of using return to send back a single value, generator functions use yield to produce a series of results over time. The function pauses its execution after yield, maintaining its state between iterations.

```
def fun(max):
 cnt = 1
 while cnt <= max:
 yield cnt # see yield function
 cnt += 1

ctr = fun(5)
for n in ctr:
 print(n)
```

```
1
2
3
4
5
```

## ✓ Testing

It involves testing small, isolated pieces of code, called “units,” to ensure they function correctly.

By catching errors early, unit testing enhances code quality, reduces bugs in production and makes programs more reliable and maintainable.

### Key points

- Each test focuses on a specific part of the code to verify its correctness.
- Errors are identified early, preventing issues from spreading.

- Unit tests improve overall code reliability and simplify future maintenance.

## Unit Testing Frameworks

Python has several frameworks to help write and run unit tests efficiently:

- unittest: Built into Python's standard library.
- pytest: Simplified syntax and powerful features.
- nose: Extends unittest for easier testing.
- doctest: Tests examples written in docstrings.

## Assert statement

Before using frameworks, it's important to understand assert statement

The assert statement is used to verify that a condition holds true in your code. If the condition is false, it raises an AssertionError, helping catch bugs early by validating assumptions during development.

`.assertEqual(a, b)` Checks if `a == b` `.assertTrue(x)` Checks if `x` is True `.assertIsInstance(a, b)` Checks if `a` is an instance of class `b`  
`.assertIsNone(x)` Checks if `x` is None `.assertFalse(x)` Checks if `x` is False `.assertIs(a, b)` Checks if `a` is `b` `.assertIn(a, b)` Checks if `a` is a member of `b` `.assertRaises(Exception)` Checks if a block of code raises an exception

## Syntax

`assert_keyword condition, message`

- condition: Expression to check. If True, execution continues. If False, raises AssertionError.
- message (optional): Displayed if the assertion fails, providing context.

```
x = 5 assert x > 0, "x must be positive"
```

### ✓ Using unittest

unittest is Python's built-in framework for writing and running tests. It helps you verify that your code behaves as expected by organizing tests into reusable test cases. To use it, start by importing library:

```
import unittest

class TestSum(unittest.TestCase): #inherit test case from import
 def test_sum_numbers(self):
 data = [100, 200, 300]
 result = sum(data)
 self.assertEqual(result, 600)

if __name__ == '__main__':
 unittest.main()
```

## ✓ Using Pytest

pytest is a Python testing framework that makes writing and running tests simpler and more flexible. It can run existing unittest test cases while offering additional benefits such as:

Needs to be installed first Pip install pytest

```
math_functions.py
def add(x, y):
 return x + y

def subtract(x, y):
 return x - y

test_math_functions.py
import math_functions

def test_add():
 assert math_functions.add(2, 3) == 5
 assert math_functions.add(-1, 1) == 0
```

```
assert math_functions.add(0, 0) == 0

def test_subtract():
 assert math_functions.subtract(5, 3) == 2
 assert math_functions.subtract(0, 0) == 0
 assert math_functions.subtract(-1, -1) == 0
```

Then cd into directory and run Pytest

- Pytest automatically finds functions starting with test\_ in files named test\_\*.py.
- assert checks if output matches expected result, failures are reported by pytest.

## ✓ Using Nose

Nose is a testing framework test runner that extends capabilities of Python's built-in unittest module. It provides a simpler syntax for writing tests and offers additional features for test discovery, parallel test execution and plugin support.

```
pip install nose
```

```
math_functions.py
def add(x, y):
 return x + y

def subtract(x, y):
 return x - y

Then create a file called main.py
def test_addition():
 assert 1 + 1 == 2

def test_subtraction():
 assert 3 - 1 == 2
```

Then run

```
nosetests main.py
```

## ✓ Using Doctest

Doctest Module is used to write tests in form of interactive Python sessions within docstrings and then automatically run those tests to verify that code behaves as expected.

Doctest Module finds patterns in docstring that looks like interactive shell commands.

Example: Let's create a file named main.py and write doctest-based tests directly in the function docstrings. These examples will automatically check if the functions work correctly.



```
def add(a, b):
 """
 Function to add two numbers.
 >>> add(2, 3)
 5
 >>> add(-1, 1)
 0
 """
 return a + b

def subtract(a, b):
 """
 Function to subtract two numbers.
 >>> subtract(5, 2)
 3
 >>> subtract(10, 5)
 4
 """
 return a - b
```

Then run:

```
python -m doctest main.py
```

## ✓ Engineering Essentials 4 – Database Systems

Database Systems are one of the most important pillars of software engineering. Almost every software application stores and retrieves data—from banking systems and hospitals to mobile apps and AI systems.

### What is a Database?

A database is an organized collection of data stored electronically.

Instead of storing data in text files, databases provide:

- Fast searching
- Efficient updates
- Security
- Multi-user access
- Backup and recovery

Example: Imagine Facebook.

Instead of

```
users.txt
posts.txt
likes.txt
comments.txt
```

they use databases that can store billions of records.

## Database vs DBMS

Database - The actual stored data. DBMS (Database Management System) - Software that manages databases.

Examples include:

- PostgreSQL
- MySQL
- SQLite
- Microsoft SQL Server
- Oracle Database

## Types of Databases

## Relational Databases (SQL) Data stored in tables.

Eg. Customers

| ID | Name |
|----|------|
| 1  | John |
| 2  | Mary |

Orders

| ID | CustomerID | Amount |
|----|------------|--------|
| 1  | 2          | 250    |

Examples of relational databases include:

- PostgreSQL
- MySQL
- MariaDB

## NoSQL Databases Not table-based.

E.g. Document

```
{
 "name": "John",
 "age": 24
}
```

There are key-value pairs name is the key and John is the value.

Examples of nosql databases include:

- MongoDB
- Redis
- Apache Cassandra

- Neo4j

## Relational Database Concepts

### Primary Key

Uniquely identifies each row.

E.g.

StudentID

### Properties

- Unique
- Never NULL
- One per table

### Foreign Key

Links one table to another.

Orders

CustomerID

points to

Customers

CustomerID

## Relationship

Customers



Orders

## Candidate Key

A column that can become the primary key.

E.g.

StudentID

Email

Both are unique.

## Composite Key

Multiple columns together form a key.

(StudentID, CourseID)

## Data Types

Common SQL data types

| Type    | Purpose        |
|---------|----------------|
| INT     | Whole numbers  |
| BIGINT  | Large integers |
| DECIMAL | Money          |

| Type     | Purpose           |
|----------|-------------------|
| FLOAT    | Scientific values |
| CHAR     | Fixed text        |
| VARCHAR  | Variable text     |
| TEXT     | Long text         |
| DATE     | Date              |
| TIME     | Time              |
| DATETIME | Date & time       |
| BOOLEAN  | True/False        |

## Structured Query Language

The language used to communicate with databases.

Four categories:

### Data Definition Language

CREATE

ALTER

DROP

TRUNCATE

### Data Manipulation Language

SELECT

INSERT

UPDATE

DELETE

## Data Control Language

GRANT

REVOKE

## Transaction Control Language

COMMIT

ROLLBACK

SAVEPOINT

## Constraints

Protect data quality.

E.g.

PRIMARY KEY

FOREIGN KEY

UNIQUE

CHECK

NOT NULL

DEFAULT

## Aggregate Functions

COUNT ( )

SUM ( )

AVG ( )

MIN ( )

MAX ( )

## Views

Virtual tables.

```
CREATE VIEW ActiveUsers
```

```
AS
```

```
SELECT *
```

```
FROM Users
```

```
WHERE Active=1;
```



## Stored Procedures

Reusable SQL programs.

### Advantages

- Faster
- Secure
- Reusable

## Triggers

Automatically execute.

E.g.

```
INSERT Employee
```

↓

```
Trigger
```

↓

```
Audit Table
```

## Transactions

A transaction is a group of operations treated as one unit of work.

E.g. in transferring money

```
Withdraw
```

↓

Deposit

Either both happen or neither.

## Normalisation

**Normalization** is the process of organizing data into tables so that:

- Data is not unnecessarily duplicated.
- Relationships between data are clear.
- Updates are easier.
- Insertions and deletions do not create problems.
- Storage is used efficiently.

Think of it as **organizing your wardrobe**:

- Shirts go in one drawer.
- Socks go in another.
- Shoes are stored separately.

If you mixed everything together, finding or updating something would become difficult. Databases work the same way.

### Why Normalise?

Suppose you build a student registration system.

Without normalisation:

| StudentID | StudentName | Course1 | Course2  | Lecturer  |
|-----------|-------------|---------|----------|-----------|
| 1         | Rudo        | Python  | AI       | Mr. Mpini |
| 2         | Mary        | Python  | Database | Mr. Mpini |

| StudentID | StudentName | Course1 | Course2  | Lecturer  |
|-----------|-------------|---------|----------|-----------|
| 3         | Peter       | AI      | Database | Mr. Mpini |

Problems:

- "Mr. Mpini" appears many times.
- "Python" appears many times.
- What if Mr. Mpini changes their name? You must update every row.
- What if a student takes three courses? You need another column (**Course3**).

This is a poor design.

### Three Types of Problems (Anomalies)

Normalization solves three major problems.

#### 1. Update Anomaly

Suppose the lecturer changes from "Mr. Mpini" to "Dr. Moyo".

Before:

| Student | Lecturer  |
|---------|-----------|
| John    | Mr. Mpini |
| Mary    | Mr. Mpini |
| Peter   | Mr. Mpini |

You must update every row.

If you miss one:

| Student | Lecturer  |
|---------|-----------|
| John    | Dr. Moyo  |
| Mary    | Dr. Moyo  |
| Peter   | Mr. Mpini |

Now the database is inconsistent.

## 2. Insert Anomaly

Suppose you want to add a new course.

But no students have enrolled yet.

Where do you put it?

You cannot insert it because the table requires student information.

## 3. Delete Anomaly

Suppose only one student takes Machine Learning.

| Student | Course           |
|---------|------------------|
| Rudo    | Machine Learning |

If Rudo graduates and you delete his record...

The course disappears from the database too.

You accidentally lost important information.

## The Normal Forms

The most common normal forms are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

Most software systems stop at **3NF** or **BCNF**.

Unnormalized Form (UNF)

Suppose a school stores data like this:

| StudentID | Name | Courses              |
|-----------|------|----------------------|
| 1         | John | Python, AI, Database |
| 2         | Mary | AI, Web Development  |

Notice:

One column contains multiple values.

This is **not relational**.

First Normal Form (1NF)

Rule: Each column must contain **only one value**. That means no lists, no arrays, and no comma separated values

| StudentID | Name | Course |
|-----------|------|--------|
| 1         | John | Python |
| 1         | John | AI     |
| 2         | Mary | AI     |

Every cell contains a single (atomic) value.

Now the table is in **1NF**.

Second Normal Form (2NF)

Now suppose we have:

| StudentID | CourseID | StudentName | CourseName |
|-----------|----------|-------------|------------|
| 1         | C1       | John        | Python     |
| 1         | C2       | John        | AI         |
| 2         | C1       | Mary        | Python     |

The primary key is:

(StudentID, CourseID)

This is called a **composite key**.

Now look carefully.

`StudentName` depends only on `StudentID`.

It doesn't depend on `CourseID`.

Similarly,

`CourseName` depends only on `CourseID`.

These are **partial dependencies (PD)**.

PD is a non-key attribute depends on only part of a composite key instead of the whole key.

### Solution (2NF)

Split the table.

Students

| StudentID | StudentName |
|-----------|-------------|
| 1         | John        |
| 2         | Mary        |

Courses

| CourseID | CourseName |
|----------|------------|
| C1       | Python     |
| C2       | AI         |

Enrollments

| StudentID | CourseID |
|-----------|----------|
| 1         | C1       |
| 1         | C2       |
| 2         | C1       |

Now every non-key attribute depends on the entire primary key of its table.

2NF achieved.

Third Normal Form (3NF)

Now consider this table.

| StudentID | Name | DepartmentID | DepartmentName   |
|-----------|------|--------------|------------------|
| 1         | John | 10           | Computer Science |
| 2         | Mary | 20           | Accounting       |

Primary key:

StudentID

Notice:

DepartmentName depends on DepartmentID.

DepartmentID depends on StudentID.

Therefore,

StudentID

↓

DepartmentID

↓

DepartmentName

This is called a **transitive dependency**. A TD is a non-key attribute depends on another non-key attribute instead of directly on the primary key.

Think of it like this:

Primary Key

↓

Column A

↓

Column B

Column B should depend directly on the primary key—not indirectly.

### Solution (3NF)

Split into two tables.

Students

| StudentID | Name | DepartmentID |
|-----------|------|--------------|
| 1         | John | 10           |
| 2         | Mary | 20           |

Departments

| DepartmentID | DepartmentName   |
|--------------|------------------|
| 10           | Computer Science |
| 20           | Accounting       |

Now:

DepartmentName depends only on DepartmentID.

Students only store the DepartmentID.

No duplication.

Boyce-Codd Normal Form (BCNF)

BCNF is a stricter version of 3NF.

Rule:

Every determinant must be a candidate key.



Most systems never need to worry about BCNF unless relationships become complex.

### Fourth Normal Form (4NF)

Removes multi-valued dependencies.

Example:

A teacher can teach many subjects.

A teacher can also supervise many clubs.

Putting both in one table creates unnecessary combinations.

Instead, create separate tables for teaching assignments and club supervision.

### Fifth Normal Form (5NF)

Deals with complex join dependencies.

Usually found in:

- Large enterprise systems
- Government databases
- Banking systems

## Denormalisation?

**Denormalisation** is a database optimization strategy where you intentionally add redundant data or group data from multiple tables into one.

Denormalization is *not* "failing to normalize." You should always normalize your database first (usually to 3rd Normal Form or 3NF) to understand your data structure. Denormalization is a deliberate, tactical step taken later to improve **read performance**.

## Why Do It? The Core Trade-Off

In a heavily normalized database, data is split across many tables. To answer a complex question, the database has to perform multiple **JOIN** operations. JOINS are CPU and memory-intensive.

By merging tables or duplicating data, you trade **write speed** and **storage space** for blazing-fast **read speed**.

| Feature                   | Normalized (3NF)                         | Denormalized                         |
|---------------------------|------------------------------------------|--------------------------------------|
| <b>Read Speed</b>         | Slower (requires complex JOINS)          | <b>Faster</b> (data is pre-combined) |
| <b>Write/Update Speed</b> | <b>Faster</b> (change data in one place) | Slower (must update duplicate data)  |
| <b>Data Integrity</b>     | High risk of anomalies minimized         | Risk of data inconsistencies         |
| <b>Storage Space</b>      | Efficient / Minimal                      | Consumes more space                  |

## Denormalisation Techniques

Here are the most common ways engineers denormalize a database:

### 1. Storing Derivable/Calculated Values

Instead of calculating a sum or average every single time a user loads a page, you calculate it once during a write and store the result.

- **Normalized:** To show an order total, the database must sum up all line items in an `Order_Items` table every time.
- **Denormalized:** You add an `order_total` column directly to the `Orders` table. It updates only when an item is added or removed.

### 2. Duplicating Attributes (Pre-joining tables)

If you almost always need a piece of data from Table B when looking at Table C, just put that data in Table C.

- **Normalized:** An `Orders` table links to a `Customers` table via `customer_id`. To show a shipping label, you must JOIN them to get the customer's name.
- **Denormalized:** You copy the `customer_name` directly into the `Orders` table. No JOIN required.

### 3. Hardcoding Codes or Lookups

If a lookup table has very few records that almost never change (like status codes or country names), you can drop the lookup table entirely.

- **Normalized:** An `employees` table has a `status_id` pointing to a `statuses` table (1 = Active, 2 = On Leave, 3 = Terminated).
- **Denormalized:** You store the text `"Active"`, `"On Leave"`, or `"Terminated"` directly in the `employees` table.

## Indexes

Think of a database table like a massive, 1,000-page textbook. If you want to find every page that mentions "acid compliance," you have two choices:

1. Scan the textbook page by page, line by line, from start to finish. (This is a **Table Scan**).
2. Flip to the alphabetical index at the back, look up "acid compliance," see that it's on pages 45, 112, and 304, and flip straight to those pages. (This is an **Index Seek**).

An index is a separate, hidden data structure (usually a **B-Tree** or a **Hash Table**) that pointers to the exact physical location of rows in your table based on the values in specific columns.

When you create an index on a column—say, `email` in a `users` table—the database takes all the emails, copies them into a highly organized, sorted data structure (most commonly a **B-Tree**), and attaches a reference pointer to each one that points back to the full row in the main table.

Because the B-Tree is sorted and balanced, finding an email doesn't take  $O(N)$  time (checking every row). It takes  $O(\log N)$  time.

- In plain English: In a table of **1 million rows**, a sequential scan might check 1,000,000 records. A B-Tree index can find the exact record in roughly **20 steps or fewer**.

### Natural Indexes with DBMS Systems

1. The Automatic Primary Key Index (Clustered Index) In MySQL's InnoDB engine, tables are indexes.

When you define a column as a PRIMARY KEY, MySQL automatically creates a Clustered Index on it.

The database physically organizes your entire table's rows on the disk based on the order of this primary key.

What if you don't define a primary key? MySQL will search for the first UNIQUE index that has no NULL values and use that as the clustered index.

What if you don't define any keys? MySQL will secretly generate a hidden, 6-byte row ID column (ROW\_ID) and build a clustered index on that. It demands a clustered index to structure its storage.

2. The UNIQUE Constraint Index Whenever you mark a column as UNIQUE (for example, guaranteeing that no two users share the same email), MySQL automatically builds a non-clustered index on that column. It does this because checking whether a value already exists during an INSERT would be painfully slow without an index.
3. Foreign Keys If you define a FOREIGN KEY constraint linking Table A to Table B, MySQL automatically creates an index on that foreign key column. This makes the JOIN operations between those related tables highly efficient right out of the box.

## ✓ MySQL Database

If you're a Python developer, you don't need to become a database administrator (DBA), but you **do** need to understand how MySQL works and how to interact with it from Python.

MySQL is an **open-source Relational Database Management System (RDBMS)**.

It stores data in **tables**, where tables are related to one another using keys.

### How MySQL Fits into a Python Application

1. Python sends SQL commands.
2. MySQL executes them.
3. Results are returned to Python.

### Installing MySQL

You'll typically install:

- MySQL Server
- MySQL Workbench (GUI)

- MySQL Shell (optional)
- MySQL Command Line Client

## Installing the Python Connector

The most commonly used library is:

```
pip install mysql-connector-python
```

Other popular libraries include:

- PyMySQL
- mysqlclient
- SQLAlchemy (ORM)
- aiomysql (for asynchronous applications)

In this tutorial we will use mysqlclient

### ✓ MySQLClient

```
Connecting Python to MySQL

import mysql.connector

connection = mysql.connector.connect(
 host="localhost",
 user="root",
 password="password",
 database="school"
)

print("Connected!")
```

```
Creating a Cursor.
Think of a cursor as your communication channel.
cursor = connection.cursor()
```

```
Executing SQL
cursor.execute("SELECT * FROM students")
```

```
Fetching one data item
student = cursor.fetchone()

Fetch all rows
students = cursor.fetchall()
```

```
Inserting Data

sql = """
INSERT INTO students(name,email)

VALUES(%s,%s)
"""

values = ("John","john@gmail.com")

cursor.execute(sql, values)

connection.commit()
```

```
Updating
cursor.execute(
```

```
"UPDATE students SET age=%s WHERE id=%s",

(25,1)

)

connection.commit()
```

```
Deleting
cursor.execute(

"DELETE FROM students WHERE id=%s",

(1,))

connection.commit()
```

```
Closing Resources
Always clean up.

cursor.close()

connection.close()
```

```
Transactions

Whenever you modify data using INSERT, UPDATE, DELETE
Always commit.

connection.commit()

So that, if something goes wrong:
```

```
connection.rollback()
```

## Object Relational Mappers

Covered under Web Developer Track

### ✓ NoSQL Databases

NoSQL stands for Not Only SQL.

Unlike relational databases, NoSQL databases do not require data to be stored in tables with a fixed schema.

Instead, data can be stored as:

- Documents
- Key-value pairs
- Graphs
- Wide columns

#### 1. Document Databases (Most Common)

Store data as JSON-like documents.

Example:

```
{
 "id": 1,
 "name": "John",
 "age": 22,
 "skills": [

```



```
 "Python",
 "Java",
 "SQL"
]
}
```

Each document is like a Python dictionary.

Popular document databases:

- MongoDB
- CouchDB

## Key-Value Databases

Everything is stored as:

Key → Value

Example:

username:1001

↓

John

Like a Python dictionary:

```
user = {
 "username": "John"
```

```
}
```

Popular examples:

- Redis
- Amazon DynamoDB

## Graph Databases

Store relationships instead of tables.

Example:

John

↓

Friend

↓

Mary

↓

Works With

↓

Peter

Useful for:

- Social networks
- Recommendation engines
- Fraud detection
- Network analysis

Popular example:

- Neo4j

## Wide-Column Databases

Instead of fixed columns, each row can have different columns.

Designed for massive amounts of data.

Popular examples:

- Apache Cassandra
- Apache HBase

## ✓ Engineering Essentials 5 - Career Preparation for Python Developers

Learning Python is only half the journey. Many developers can write code, but employers hire developers who can **demonstrate skills, communicate effectively, collaborate using Git, solve real-world problems, and continue learning.**

## ✓ Portfolio Building

Employers trust projects more than certificates. A portfolio proves you can build software.

Instead of saying

"I know Python" Show them "A Python system for ABC"

Quality matters more than quantity. So dont be obsessed with having too many. Focus on polised projects

The first most important project should be portfolio website. Its like your CV online but without the much sensitive details.

Then for projects listed on your projects make sure you have:

- Screenshots
- Live demo link
- A readme file which includes Project Name, Description, Features, Technologies, Installation Instructions

## Your resume

Keep it to **one page** if you have less than 5 years of experience.

Make sure your resume details

- Your Name
- Contact Information
- Professional Summary
- Technical Skills
- Projects
- Experience
- Education
- Certifications
- Achievements

Make sure you have an ATS-Friendly Resume. Many companies use Applicant Tracking Systems.

Avoid tables, images, icons, and fancy layouts

Use relevant keywords like Python, Django, REST API, Docker, PostgreSQL, Git, FastAPI, Linux, Cloud

## Preparing for Interviews

Most Python interviews follow the pattern of and HR interview (now mostly AI interview), and a coding exercise

## GitHub Profile Optimisation

Recruiters often review GitHub before making decisions.

So make sure you:

1. Use a professional username
2. Pin your best projects
3. Create a profile README which details your name, skills, what you're currently learning, contact details
4. Commit regularly. Small commits every week are better than one massive upload.

## LinkedIn Optimisation

Recruiters search LinkedIn daily.

So make sure you have:

1. A clear face
2. Neutral background
3. In the about include: Years of experience, Technologies, Interests, Projects, Career goals
4. Add your experience (if you dont have experience, add projects like experience)
5. On skills, aim for 30–50 relevant skills, including:
6. Connect with developers, recruiters, engineering managers, and hiring managers

## Freelancing

Freelancing helps you build experience before landing a full-time role.

Popular platforms include:

- Upwork
- Fiverr
- Freelancer

- Toptal (more experienced developers)
- PeoplePerHour

Beginner friendly services for a freelancer include:

- Building web applications
- Building REST APIs
- Automate Excel tasks
- Web scraping